LoopCoder-v2：仅循环一次以实现高效的测试时计算扩展

Jian Yang[1] Shawn Guo[2] Wei Zhang[2] Tianyu Zheng[2] Yaxin Du[2] Haau-Sing Li[2] Jiajun Wu[2]
Yue Song[2] Yan Xing[2] Qingsong Cai[2] Zelong Huang[2] Chuan Hao[2] Ran Tao[2] Xianglong Liu[1]
Wayne Xin Zhao[4] Mingjie Tang[2] Weifeng Lv[1] Ming Zhou[3] Bryan Dai[2]
[1]Beihang University [2]IQuest Research [3]Langboat [4]Renming University of China
{jiayang}@buaa.edu.cn

HuggingFace: https://huggingface.co/Multilingual-Multimodal-NLP/LoopCoder-V2

arXiv:2606.18023v1 [cs.LG] 16 Jun 2026

摘要

循环变换器通过反复应用共享块来扩展潜在计算，但顺序循环会随着循环次数的增加而增加延迟和键值缓存内存。并行循环变换器通过跨循环位置偏移和共享键值的门控滑动窗口注意力来缓解这一成本，使循环次数成为一个实用的设计选择。因此，我们从增益-成本视角研究并行循环变换器的循环次数选择：额外的循环可能细化表示，但跨循环位置偏移也会在每个循环边界引入位置不匹配。我们通过从头开始在 18 万亿个标记上训练 LoopCoder-v2（一个具有不同循环次数的 7B 并行循环变换器编码器系列），随后进行匹配的指令微调和评估，来实例化这项研究。经验上，双循环变体在代码生成、代码推理、智能体软件工程和工具使用基准上比无循环基线带来了广泛的增益，将 SWEbench Verified 从 43.0 分提高到 64.4 分，Multi-SWE 从 14.0 分提高到 31.0 分。相比之下，具有三个或更多循环的变体出现退化，揭示了强烈的非单调循环次数效应。我们的诊断表明，第 2 个循环提供了主要的生产性细化，而后续循环产生递减的、振荡的更新和降低的表示多样性。由于跨循环位置偏移引起的不匹配大致保持固定，而细化增益缩小，偏移成本日益占主导地位。这种增益-成本权衡解释了并行循环变换器在两个循环处的饱和，并为循环次数选择提供了诊断依据。

1. 引言

循环大语言模型（LLM）[16, 17] 已成为一种有前景的方法，可在不按比例增加参数数量的情况下扩展语言模型的有效计算深度。循环大语言模型并非堆叠多个不同的层，而是

（大语言模型）反复应用共享的变换器（Transformer）模块，使相同参数能够执行多轮潜在空间计算 [5, 9, 18]。该设计对测试时计算扩展尤为有利，无需生成辅助推理标记即可实现额外的内部细化 [8]。近期研究表明，此类循环深度大语言模型可逼近更深的非循环变换器，并随着推理时计算量的增加而提升推理性能 [8, 14, 19]。

尽管前景广阔，标准的顺序循环难以高效部署：每增加一次循环都需要再次遍历共享模块，并引入循环特定的键值缓存

状态，导致延迟和内存随循环次数增长 [15]。并行循环变换器（Parallel Loop Transformer，PLT）[16] 通过两种互补机制缓解这一瓶颈：跨循环位置偏移（crossloop position offsets，CLP），打破循环间的顺序依赖并实现并行循环执行；以及共享键值门控滑动窗口注意力（共享键值门控滑窗注意力机制，G-SWA），使缓存占用在不同循环次数下几乎保持不变。然而，降低循环成本本身并不能确定最佳工作点。在 PLT 中，循环次数成为部署时的设计选择：循环过少可能未充分利用模型的细化能力，而循环过多可能引入冗余或有害计算。穷举训练和评估每个候选循环次数代价高昂，且难以解释特定设置成功或失败的原因。这引出一个更具诊断性的问题：能否通过检查每个循环内部的贡献来识别 PLT 的饱和点？

为探究这一问题，本文从增益与代价的视角审视并行循环 Transformer（Parallel Loop Transformer，PLT）的循环计数行为（图 1）。在增益方面，额外循环仅在执行有意义的细化时才有价值：连贯地更新隐藏状态、改变信息路由，并促使模型输出分布发生偏移。因此，本文追踪各循环间的隐藏状态动态、注意力演化及输出分布偏移。在代价方面，循环级并行（CLP）通过以相邻状态的偏移依赖取代直接的同一词元循环来实现并行，这可能在每个循环边界引入位置错配。本文从模型的隐藏状态中量化这一错配，并将其与每个循环的边际增益相关联。所有比较均在匹配的训练、指令微调和评估设置下进行，确保由此产生的循环间差异反映的是循环计数的影响，而非协议变化所致。

我们在 LoopCoder-v2 上实例化了这项研究，这是一个 7B 的并行循环变换器（PLT）编码器，从零开始在 18T 个混合文本和代码数据的标记上训练，随后进行指令微调。比较具有 ($R \in \{1, 2、3、4$ 的匹配循环计数变体，我们观察到一个强烈的非单调趋势：双循环模型在非循环基线上有广泛改进，包括在 SWE-bench Verified 上从 43.0% 提升到 64.4%，而三循环模型在许多任务上出现退化，在 SWE-bench Verified 上下降到 27.6%。这种模式表明额外的 PLT 循环可能变得有害，促使我们对边际细化增益和 CLP 引起的偏移成本进行逐循环分析。

我们的贡献如下：

1. **并行循环变换器（PLT）循环计数选择的增益-成本视角。**我们将 PLT 循环计数选择表述为额外循环带来的边际细化与 CLP 在循环边界引入的结构成本之间的权衡。

2. **并行循环变换器（PLT）饱和的逐循环诊断。**我们分析了跨循环的隐藏状态动态、注意力演化和输出分布偏移，并定义了一个内在偏移成本 $\Omega^{(r)}$ 来量化 CLP 引起的位置不匹配。我们的分析表明，第二个循环提供了主要的有效细化，而后续循环产生递减且日益振荡的更新。

3. **使用 7B 并行循环变换器（PLT）编码器的大规模实证研究。**我们从零开始训练 LoopCoder-v2，在 18T 个混合文本和代码数据的标记上使用不同的循环计数，并在匹配的训练、指令微调和评估设置下比较循环计数变体。双循环模型在代码生成、代码推理、智能体软件工程和工具使用基准上广泛改进，而额外循环通常会导致退化。

2. 预备知识与问题形式化

本节对本文研究的循环次数选择问题进行形式化。首先回顾标准的循环变换器（Looped Transformer），其中额外的循环增加了潜在的计算深度，但也带来了顺序推理成本。随后引入并行循环变换器（PLT），

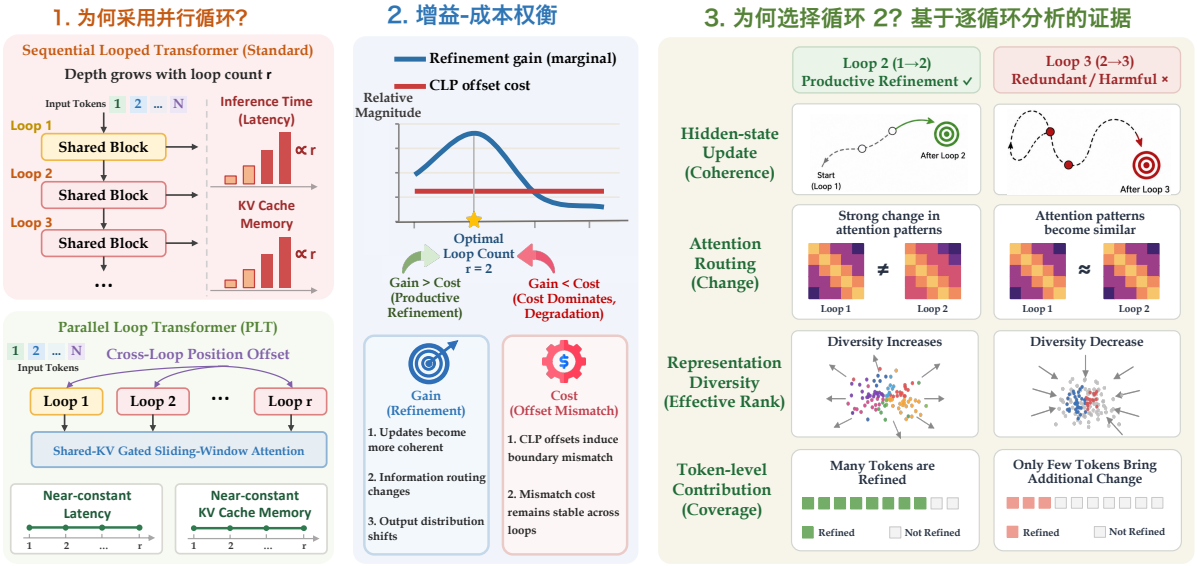


图 1. PLT 循环次数选择概览。左：标准顺序循环的延迟和键值缓存内存随循环次数增加，而 PLT 利用跨循环位置偏移和共享键值 G-SWA 使两者几乎保持不变。中：每个新增循环在边际细化增益与 CLP 引起的偏移失配之间进行权衡。增益早期达到峰值后逐渐减小，而偏移成本大致稳定，使得 $r = 2$ 成为优选工作点。右：逐循环诊断解释了这一选择。循环 2 表现出连贯的隐藏状态移动、变化的注意力路由、增加的表示多样性以及广泛的令牌级细化，而循环 3 则更具冗余性且产出较低。

该方法通过并行循环执行和共享键值注意力降低了这一成本，同时改变了跨循环迭代的信息流。最后，比较两种设置，并将 PLT 循环次数选择形式化为增益-成本权衡：每个额外循环可能提供有用的表示细化，但在 PLT 中也会引入偏移引起的位置失配。

2.1. 循环变换器

循环变换器（Looped Transformer）将多个不同层的深层堆叠替换为单个共享块 f_θ，该块由 L 层组成，并重复应用 [5, 9] 次。给定输入标记 x，它展开递归过程

$$\mathbf{h}^{(0)} = \mathrm{Embed}(x), \qquad \mathbf{h}^{(r)} = f_\theta\left(\mathbf{h}^{(r-1)}\right), \quad r = 1, \ldots, R, \qquad \mathit{logits} = \mathrm{Head}\left(\mathbf{h}^{(R)}\right),$$

其中 $\mathbf{h}^{(r)}$ 是第 r 个隐藏状态，R 是循环次数。在每次迭代中重用相同的参数，使得有效深度 $R \cdot L$ 随 R 增长，而参数数量 N 保持不变，因此循环模型在较小的参数预算下实现了深度计算，在深度敏感任务上可与大得多的非循环模型相媲美 [18]。

然而，这种深度带来的推理成本随 R（扩展（表 1））。递归是严格顺序的：$\mathbf{h}^{(r)}$，在 $\mathbf{h}^{(r-1)}$, so R 之前无法计算，循环需要 R 次连续通过 f_θ，并将墙钟延迟乘以 R。内存成本同样急剧增长：标准实现会在每个循环的每一层缓存键和值，因此 KV 缓存占用达到 $O(R \cdot L \cdot S \cdot d)$，对于序列长度 S 和宽度 d，是单次传递的 R 倍 [15]。因此，延迟和内存都随着每次循环的增加而增长，使得深度循环推理在延迟或内存受限的部署中不切实际。

2.2. 并行循环变换器

并行循环变换器（Parallel Loop Transformer，PLT）[16] 是一种具有代表性的设计，旨在降低这两项成本。PLT 引入了两种独立机制：共享的首循环 KV 缓存，通过门控滑动窗口注意力访问，从而限制内存；以及跨循环位置偏移，消除了循环之间的顺序依赖。

高效表示增强（KV 共享 + 门控滑动窗口注意力（G-SWA））。来自第一个循环的 KV 缓存 $K_{\text{share}}, V_{\text{share}} = \text{KV}(\mathbf{h}^{(1)})$ 与所有后续循环共享，以保持总 KV 缓存内存为 $O(L \cdot S \cdot d)$，无论 R 如何。在非首循环中，每个注意力层对两个注意力输出执行门控融合：

$$\tilde{y}^{(r)} = g \odot y^{(r)}_{\text{global}} + (1-g) \odot y^{(r)}_{\text{local}}, \quad g = \sigma\big(f_{\text{gate}}(\text{RMSNorm}(\mathbf{h}))\big), \tag{1}$$

其中 $y^{(r)}_{\text{global}}$ 是对来自循环 1 的冻结 $K_{\text{share}}, V_{\text{share}}$ 的全上下文注意力，$y^{(r)}_{\text{local}}$ 是对当前循环 KV 的宽度为 $w = 64$ 的滑动窗口注意力。门控 f_{gate} 是应用于 RMSNorm 归一化层输入的头级线性层，为每个头产生一个标量。

跨循环并行（Cross Loop Parallelism (CLP) 偏移）。在每个循环 $r \geq 2$ 之前，前一个循环的隐藏状态向右移动一个标记位置，并加回到输入中：

$$B^{(r)} = \text{Embed}(x) + \text{shift}\left(\mathbf{h}^{(r-1)}\right), \qquad \mathbf{h}^{(r)} = f_\theta\left(B^{(r)}\right), \tag{2}$$

其中 $\text{Embed}(x)$ 是输入 x，$\mathbf{h}^{(r-1)}$ 的标记嵌入序列，循环-$(r-1)$ 隐藏状态，以及一位右移 $\text{shift}(\mathbf{h}^{(r-1)})_i = \mathbf{h}^{(r-1)}_{i-1}$（$\mathbf{h}^{(r-1)}_0 = \mathbf{0}$）。和 $B^{(r)}$ 是循环 r 的输入，共享块 f_θ 将其映射到 $\mathbf{h}^{(r)}$。这消除了连续循环中相同索引状态之间的直接位置依赖，因此标记 x_i 的第 r 次循环可以与标记 x_{i-1} 的第 $(r+1)$ 次循环在单次前向传递中并发计算，实现接近单次传递的墙钟延迟。

偏移的信息流后果。标记 x_i 在循环 $r \geq 2$ 处接收的输入是其自身嵌入与标记 x_{i-1} 的循环-$(r-1)$ 隐藏状态的混合，而非其自身的。因此，偏移引入了每个标记的位置不匹配：标记 x_i 在循环 r 处可用的状态反映的是 x_{i-1} 所见的上下文，而非 x_i 自身。这种位置不匹配是 PLT 引入的核心信息约束。

2.3. 循环次数选择作为增益-成本权衡

表 1 对比了两种设置：PLT 消除了顺序依赖并冻结了第一循环的 KV 缓存，因此延迟和内存都不随 R 扩展。这些节省使得在推理时增加循环次数变得可行，我们采用 PLT 作为本文其余部分的实验框架。

然而，这种效率并非没有代价：CLP 偏移使每次增加的循环都耦合了表征增益，即进一步的潜在细化，与标准循环从未产生的位置不匹配成本。这些对立力量如何平衡决定了 PLT 表现最佳的循环次数，即本文其余部分通过逐循环可解释性分析（第 4 节）量化的增益-成本权衡。

表 1. 顺序循环与并行循环变换器（PLT）的对比，其中 C_{block} 为通过共享块一次的成本。PLT 使延迟和内存均与循环次数无关 R 。

	Sequential loop	**PLT**
Execution	sequential	parallel, single pass
Latency	$O(R\,C_{\text{block}})$	$\approx C_{\text{block}}$
KV-cache	$O(RLSd)$	$O(LSd)$
Inter-loop input	$\mathbf{h}^{(r-1)}$	$\text{Embed}(x) + \text{shift}(\mathbf{h}^{(r-1)})$

3. 并行循环变换器分析

本文在两个互补的分辨率上分析 PLT 的循环次数行为。首先采用宏观视角：在严格匹配的协议下，改变循环次数并测量其对下游性能的影响，从而确定循环次数的重要性及最佳工作点。随后采用微观视角：通过一系列逐循环诊断透镜，揭示模型内部计算，解释宏观曲线为何呈现该形状。第一种视角揭示循环累积时发生的情况，第二种视角揭示每个循环实际如何产生该结果。

3.1. 宏观视角：循环次数效应

本文首先隔离循环次数本身的影响。在架构、数据和调优保持不变、仅改变 R 的条件下，探究在推理时投入更多潜在循环时下游任务性能如何响应，这一宏观现象正是第 3.2 节微观分析所要解释的内容。

模型配置。所有分析均在配备 PLT 机制、窗口大小为 $w = 64$ 的 G-SWA 及首循环 KV 共享的 70 亿参数稠密变换器上进行，该机制统一应用于所有循环。完整架构配置见附录 B。

训练协议。模型在一个内部去重后的文本与代码混合数据上训练，总计 18T 个词元，文本与代码词元比例为 1:1。代码部分涵盖 100 多种编程语言，其详细构成见表 6。训练与推理的循环次数全程保持一致：以 $R = r$ 训练的模型在 $R = r$. 下进行评估。我们使用 Adam 优化器，参数为 $\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-15}$ ，权重衰减 0.1，梯度裁剪范数为 1.0。学习率为 $\eta = 4 \times 10^{-4}$ ，采用余弦衰减调度，并在前 5% 的训练步中进行线性预热。所有运行均使用 bf16 混合精度与梯度检查点。总体而言，本工作中训练不同循环次数的 LoopCoder-v2 共消耗 100 万 GPU 小时。

训练基础设施。我们在定制的 Megatron-LM 栈上训练 PLT，该栈原生支持权重绑定的循环展开。 R 在 L 层共享块上的循环被展开为 $R \cdot L$ 个调度层，但只有第一个循环实例化参数；后续循环针对相同模块的引用执行，因此参数数量、优化器状态和检查点占用始终保持为单个 L 层块的大小，与 R 无关。为了在流水线并行下实现无通信的权重共享，虚拟流水线布局将给定层的所有 R 个实例放置在同一流水线阶段。第一个循环执行标准全注意力并缓存其键和值；每个后续循环对每层发出两次 Transformer-Engine 注意力调用，一次是对当前循环键值的宽度为 w 的滑动窗口注意力

以及一次对冻结的第一循环缓存的全注意力，两者通过逐头门控（方程 1）结合，该门控初始化为零，以实现均衡的局部/全局混合。由于跨循环偏移复用了第一循环缓存和词元嵌入，这些张量被分离，其梯度通过自定义反向钩子累积，从而在（虚拟）流水线调度下保持正确的自动微分。

评估协议。四个模型均采用相同的监督微调方案，在 600 万条指令微调样本上进行训练，并在各自匹配的训练/推理循环次数下进行评估：基线模型为 $R = 1$ 次，三个 PLT 模型分别为 $R = 2, R = 3$ 次和 $R = 4$ 次。我们在涵盖代码生成、多语言代码、代码推理、数据科学与 SQL、智能体软件工程以及通用工具使用的广泛外部基准套件上评估每个模型，采用各基准的标准协议和度量。我们报告每个模型的最终监督微调检查点。结果以及与一系列开源和专有模型的比较见第 4.1 小节。

3.2. 微观视角：逐循环诊断透镜

宏观视角确立了循环次数具有决定性作用，但未解释其原因：下游准确率是一个单一标量，无法说明模型内部发生了什么。为了揭示计算过程，我们深入模型内部，探究每个循环的贡献以及 CLP 偏移如何调节该贡献。我们不依赖单一探针，而是通过三个互补的透镜进行三角验证，每个透镜审视前向传播的不同阶段。只有当各透镜结论一致时，才认定某种机制成立。隐藏状态动力学考察表征的细化过程，注意力热图演化考察信息路由方式，输出分布偏移考察细化产生的预测变化。除了这些透镜捕捉到的增益方面，我们还通过内在偏移代价来度量代价方面，该代价直接从模型自身状态量化 CLP 引起的位置不匹配。

3.2.1. 逐循环隐藏状态动力学

我们在每个循环步骤跟踪四个统计量 r，以刻画表征更新的性质和幅度。

步长与角度变化。步长 $\delta^{(r)} = \left\| \mathbf{h}^{(r)} - \mathbf{h}^{(r-1)} \right\|_2$ 度量循环 r 处隐藏状态更新的幅度，其中 $\|\cdot\|_2$ 表示全文使用的欧几里得（L_2）范数。角度变化表示为：

$$\cos\theta^{(r)} = \frac{\left\langle \mathbf{h}^{(r)} - \mathbf{h}^{(r-1)},\ \mathbf{h}^{(r-1)} - \mathbf{h}^{(r-2)} \right\rangle}{\left\| \mathbf{h}^{(r)} - \mathbf{h}^{(r-1)} \right\|_2 \left\| \mathbf{h}^{(r-1)} - \mathbf{h}^{(r-2)} \right\|_2} \tag{3}$$

其中 $\cos\theta^{(r)}$ 是两次连续更新之间的更新方向对齐程度：$\cos\theta^{(r)} \approx 1$ 表示连续循环沿同一方向持续细化，$\cos\theta^{(r)} \approx 0$ 表示正交更新，$\cos\theta^{(r)} < 0$ 表示方向反转，即振荡而非收敛的细化 [12]。

有效秩与不动点间隙。有效秩

$$\mathrm{erank}(\mathbf{h}^{(r)}) = \exp\left(-\sum_i \bar{\sigma}_i \log \bar{\sigma}_i \right), \tag{4}$$

其中 $\bar{\sigma}_i$ 是 $S \times d$ 隐藏状态矩阵的归一化奇异值（在均方根归一化后的状态上计算，因此该测度与尺度无关），衡量了几何多样性

循环 r 处的令牌表示。表示多样性从嵌入层到早期循环急剧上升，并在循环 2 达到峰值。随后的下降表明，后续循环开始收窄表示子空间而非丰富它，削弱了模型维持令牌特定信息的能力 [3]。不动点间隙直接测度当前状态偏离共享块不动点的程度，为残差细化能力提供标量总结，如下所示：

$$\Delta_{\mathrm{FP}}^{(r)} = \left\| \mathbf{h}^{(r)} - f_\theta\left(\mathbf{h}^{(r)}\right) \right\|_2 . \tag{5}$$

固有偏移代价。在循环语言模型（CLP）机制下，循环 $r \geq 2$ 的输入为 $B^{(r)} = \text{嵌入}(x) + \text{偏移}(\mathbf{h}^{(r-1)})$（方程 2），因此词元 i 接收其邻居 $i-1$ 在循环 $(r-1)$ 的隐藏状态，而非其自身。这种替换对输入信号的扭曲程度直接取决于相邻词元表示在该循环边界处的差异程度。因此，我们将循环 r 处的固有偏移代价定义为上一循环中相邻词元表示之间的平均欧氏距离。该逐循环标量 $\Omega^{(r)}$ 可直接从大语言模型的相邻隐藏状态计算得出。

$$\Omega^{(r)} = \frac{1}{S} \sum_{i} \left\| \mathbf{h}_i^{(r-1)} - \mathbf{h}_{i-1}^{(r-1)} \right\|_2 \tag{6}$$

其中 S 为序列长度，较小的 $\Omega^{(r)}$ 表明表示已开始同质化，使偏移几乎无损。经验上 $\Omega^{(r)}$ 在各循环间几乎恒定：相邻词元表示在每个循环边界处保持相当程度的异质性，因此循环语言模型偏移在每次迭代中施加近似固定的位置代价。由于额外循环的收益随深度迅速减小，这一固定代价在每个循环净效应中所占比例越来越大，因此超过少量循环后，偏移惩罚逐渐超过不断缩小的收益。固定偏移代价与递减循环收益之间的这种相互作用是第 4.2 节研究的核心机制。

3.2.2. 跨循环注意力热图演化

注意力模式记录了模型在每个循环中如何在令牌位置之间分配信息，其演变揭示了连续循环是专门化、参与不同的令牌关系子集，还是退化为注意力冗余。

逐循环注意力统计量。我们跟踪每个循环的两个标量统计量。注意力熵对于头 h 在查询位置 q，

$$\mathcal{H}_q^{(r,h)} = -\sum_{k=1}^{S} A_{qk}^{(r,h)} \log A_{qk}^{(r,h)}, \tag{7}$$

其中量化了头在循环 r 处是全局分散还是局部聚焦。循环间 KL 发散

$$D_{\mathrm{KL}}^{(r)} = \frac{1}{HS} \sum_{h,q} \mathrm{KL}\left(A_q^{(r,h)} \,\|\, A_q^{(r-1,h)} \right) \tag{8}$$

它衡量了连续循环之间注意力分布的变化程度。$D_{\mathrm{KL}}^{(r)}$ 快速衰减至零表明信息路由已有效冻结，后续循环无论其隐藏状态更新如何 [11]，都不再增加新的注意力级计算。

注意力头多样性。为了衡量注意力头是保持专门化还是坍缩为冗余路由，我们在每个循环中计算每个查询位置处 H 每头注意力分布的有效秩、头注意力向量 $H \times S$ 矩阵的归一化奇异值谱的熵，以及头之间的平均成对余弦相似度。有效秩下降，等价于头相似度上升，表明头越来越以相同方式路由信息：这是隐藏状态有效秩收窄在注意力层面的类比。

门控滑动窗口注意力（G-SWA）中的局部与全局注意力。在并行循环 Transformer（Parallel Loop Transformer，PLT）中，每个非首次循环的注意力机制是局部滑动窗口分量（基于当前循环的键值缓存）与全局分量（基于冻结的循环 1 键值缓存）的门控混合，如方程 1 所示。我们分别跟踪各头与各位置的平均门控值 $\bar{g}^{(r)}$。根据我们的约定（方程 1）$\bar{g}^{(r)}$ 表示赋予全局分支（冻结的循环 1 键值缓存）的权重），因此 $\bar{g}^{(r)} \to 1$ 表示几乎完全依赖循环 1 的全局表示，而 $\bar{g}^{(r)} \to 0$ 表示依赖新鲜的局部上下文。若门控值始终显著高于 0.5 且跨循环变化很小，则表明后续循环持续利用同一冻结的全局缓存，而非构建性质上全新的上下文。

3.2.3. 跨循环的输出分布偏移

将输出头应用于中间隐藏状态 $\mathbf{h}^{(r)}$，得到词元概率分布 $p^{(r)} = \mathrm{Softmax}(\mathrm{Head}(\mathbf{h}^{(r)}))$。跨循环跟踪 $p^{(r)}$ 可以揭示模型是逐步细化粗略的初始预测，还是其输出分布出现振荡或停滞 [3]。

逐循环输出偏移度量。我们采用三种度量。对数透镜秩（Logit Lens Rank）[11]，即真实下一个词元在 $p^{(r)}$ 下的秩，指示循环 r 是否使模型更接近正确预测。秩在 r 上单调递减构成迭代细化的最清晰特征。输出分布的循环间 KL 散度，

$$\Delta p^{(r)} = \mathrm{KL}\left(p^{(r)} \,\|\, p^{(r-1)}\right), \tag{9}$$

用于度量每个循环步骤的预测变化。输出熵 $\mathcal{H}(p^{(r)})$ 跟踪模型在循环累积过程中做出预测的置信度。

细化集中在哪里。除总体趋势外，我们探究上下文后细化（$r \geq 2$）如何在各细化循环之间分配，通过三种独立视角度量每个循环的份额：其输出偏移的幅度 $\Delta p^{(r)}$、其执行的注意力重路由量 $D_{\mathrm{KL}}^{(r)}$，以及其作为峰值贡献循环的词元比例 $r^* = \arg\max_r \Delta p^{(r)}$。第一个循环将嵌入映射到上下文状态，主导无条件分布，因此被排除以隔离细化过程。

4. 逐循环可解释性分析

4.1. 主要结果

表 2 将我们的模型与一系列开源和专有系统在代表性基准子集上进行了比较。有两个观察结果尤为突出。首先，性能与循环次数呈强烈的非单调关系：仅增加一次循环（$R = 2$）相比无循环基线有显著提升，而再增加一次循环（$R = 3$）则出现性能回退，通常低于基线。其次，仅增加一次循环使我们的 7B 模型与规模大得多的系统相比具有惊人的竞争力，尤其是在 SWE-bench Verified 上，其准确率达到 64.4%，超越了 30B 至 72B 的开源模型，并接近

表 2. 代码生成与智能体/工具使用基准上的比较。每列最佳以粗体显示；我们的 7B 模型以阴影标示。单次额外循环（R= 2）在智能体任务上与规模大得多的系统相比具有竞争力，而第二次额外循环（R= 3）则出现性能回退。平均值为可用基准上的均值（标记为 – 的条目被排除）。HE+：HumanEval+；MultiPL-E：多语言平均值；BCB：BigCodeBench-Full；LCB：LiveCodeBench；SWE：SWE-bench Verified；SWE-M：SWE-bench Multilingual；TB-v1/v2：Terminal-Bench；M2W：Mind2Web；BFCL：工具使用（v3）。

Model	HE+	MultiPL-E	BCB	LCB	SWE	SWE-M	TB-v1	TB-v2	M2W	BFCL	Avg.
Small Open models, ≤ 32B											
DeepSeek-Coder-V2-Lite-Instruct	75.6	71.5	37.8	19.4	0.0	0.0	5.0	0.0	26.7	–	26.2
Qwen2.5-Coder-7B-Instruct	81.7	75.4	37.8	18.9	0.0	0.0	6.3	0.0	38.4	54.2	31.3
Seed-Coder-8B-Instruct	75.6	75.1	44.6	22.3	0.0	0.0	7.5	2.5	38.2	–	29.5
Qwen2.5-Coder-14B-Instruct	59.8	78.8	47.0	24.6	0.0	0.0	8.8	0.0	42.7	59.9	32.2
Qwen2.5-Coder-32B-Instruct	86.6	79.6	48.0	27.4	0.0	0.0	5.0	4.5	32.5	62.3	34.6
Large open models											
Qwen3-235B-A22B-Instruct-2507	91.5	87.9	47.4	51.8	45.2	–	15.0	13.5	49.0	71.2	52.5
Kimi-Dev-72B	86.0	80.3	45.4	40.0	60.4	–	–	2.3	–	55.5	52.8
Kimi-K2-Instruct-0905	89.6	85.7	49.8	53.7	69.2	33.5	44.5	27.8	53.4	70.3	57.8
Qwen3-Coder-480B-A35B-Instruct	92.7	87.4	49.4	53.9	67.0	32.7	37.5	23.6	54.0	68.7	56.7
DeepSeek-V3.2	88.4	85.8	48.1	83.3	73.1	37.4	23.8	46.4	47.2	68.8	60.2
GLM-4.7	79.9	69.0	45.7	84.9	73.8	–	36.3	41.0	53.7	64.8	61.0
Proprietary models											
GPT-5.1	90.0	86.2	46.8	87.0	76.3	–	35.0	47.6	55.1	64.4	65.4
Claude-Opus-4.5	93.3	91.0	**53.3**	87.1	**80.9**	**50.0**	**47.5**	**59.3**	57.9	**78.9**	**69.9**
Gemini-3-Pro	**94.5**	**91.2**	47.1	**91.7**	76.2	42.7	46.3	54.2	**60.3**	78.2	68.2
Ours (7B)											
Baseline (R=1)	81.1	69.5	40.1	27.4	43.0	14.0	26.3	11.2	35.3	32.2	38.0
LoopCoder-v2 (R=2)	**84.1**	**73.9**	**46.1**	**35.4**	**64.4**	**31.0**	**34.2**	**21.0**	**34.5**	**40.1**	**46.5**
LoopCoder-v2 (R=3)	75.0	69.8	43.3	28.6	27.6	11.0	30.0	12.2	35.1	36.3	36.9
LoopCoder-v2 (R=4)	76.8	67.3	40.8	24.5	22.4	9.3	26.3	9.0	41.4	39.5	34.3

480B 规模的模型。同样的配置在智能体软件工程基准测试（SWE-bench-CC）上也达到了 33.4%，证实了第二轮循环的增益可以迁移到留出的智能体设置中。非单调曲线在额外一轮循环后达到峰值，这一现象的表征起源将在本文其余部分进行探究。

4.2. 综合：环路贡献与偏移代价

本文报告了如第 3.2.1 小节所述、结合图 2 展示的每循环隐藏动态，以及
图 3。随后，我们借助图 4 和图 5 可视化第 3.2.2 小节中提到的注意力头演化。最后，我们借助图 6 和图 7 展示第 3.2.3 小节中提到的输出分布偏移。

第 2 循环是产生有效精化的主要场所。第一个循环执行最大的变换，将嵌入映射到上下文状态，但在精化循环中，第二个循环带来了最有意义的变化：它产生了最大的循环间注意力发散 $D_{\mathrm{KL}}^{(2)}$ 和最大的每循环输出偏移 $\Delta p^{(2)}$，并且有效秩在此达到峰值。因此，表征多样性在第 2 循环处达到最大，而每一个更深的精化循环只会使其收窄。

第 2 循环之后：收益递减且非生产性。超过第二个循环后，每次迭代的边际贡献急剧下降： $\Delta p^{(r)}$ 和 $D_{\mathrm{KL}}^{(r)}$ 大幅下降，有效秩从第 2 循环的峰值下降，表明更深的循环收窄了表征子空间而非丰富它。注意力头在路由层面也反映了这一点：它们的多样性下降，并且

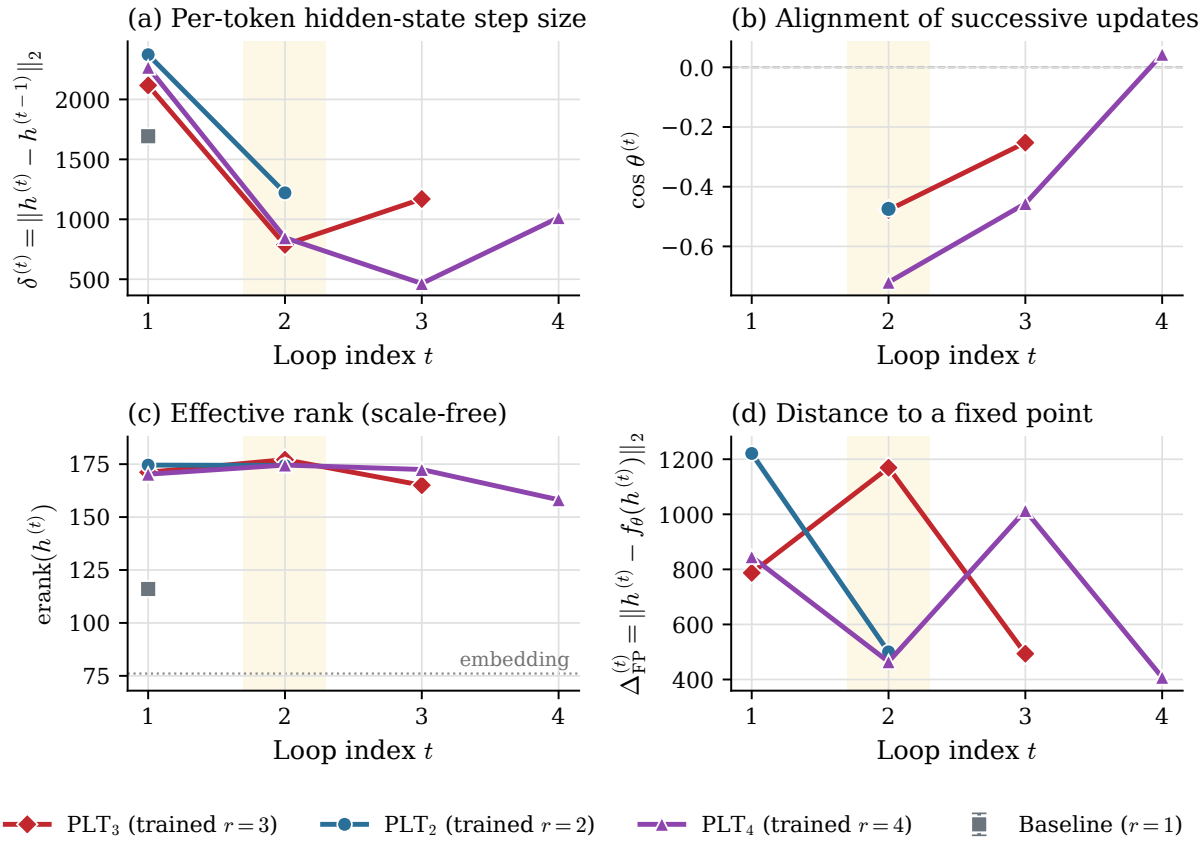


图 2. 步长 $\delta^{(r)}$（左上）、角度变化余弦 $\theta^{(r)}$（右上）、尺度无关有效秩 erank $(\mathbf{h}^{(r)})$（左下）和不动点间隙 $\Delta_{\mathrm{FP}}^{(r)}$（右下）作为循环索引 r. 的函数。线条：PLT 2 /PLT 3 /PLT 4 （训练 R= 2、3、4）；基线（ R= 1）在定义处显示。阴影带为 500 个样本上的 95% 置信区间（通常比标记更窄）；图 (c) 中的虚线标记嵌入。有效秩在第 2 循环达到峰值，并在每个更深的循环中下降；连续更新在精化循环中呈振荡性（ $\cos\theta^{(r)} < 0$），步长缩小至中间深度最小值，然后在最终（输出）循环反弹。

它们从循环到循环变得越来越冗余（图 4）。第 2 循环之后的隐藏状态更新要么接近惰性，要么呈振荡性：连续更新方向反转（ $\cos\theta^{(r)} < 0$), so 额外计算反映的是非收敛运动或最终读出，而非真正的精化。在四循环模型中，这在中间的额外循环处最为明显，它充当近乎死寂的直通通道，而最终循环仅重新读取预测（图 7）。

跨循环并行（CLP）偏移是每个循环的固定开销。固有偏移代价 $\Omega^{(r)}$（ 方程 6) 在各循环间近似恒定：CLP 偏移在每个循环边界注入可比的错位。由于每个额外循环的收益迅速递减，这一固定代价在净效应中占比不断上升，因此超过第二个循环后，偏移惩罚日益压过缩小的增益。结合峰值后有效秩的收窄，这为性能在 $R = 2$ 处达到峰值并随进一步循环而下降提供了机制性解释。

4.3. 显式与隐式思维链互补

上述循环计算可视为一种隐式思维链：模型在表示空间中跨循环进行迭代精化，而不发出任何

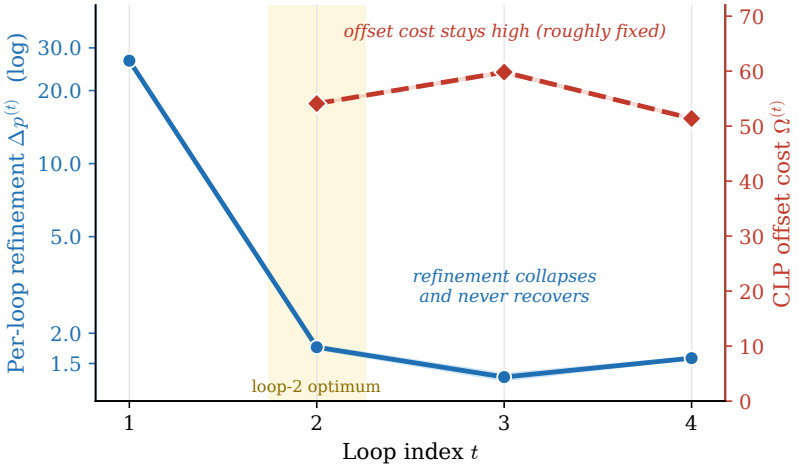


图 3. 增益 – 代价剪刀差（PLT4）。每循环精化增益 $\Delta p^{(r)}$（输出分布 KL；左轴，对数）在循环 2 后崩溃且不再恢复，而固有 CLP 偏移代价 $\Omega^{(r)}$（方程 6；右轴）保持高位且大致固定。在每个额外循环处，偏移代价超出每循环增益 30 – 45 ×，因此固定偏移税在循环 2 之后主导缩小的精化。500 个样本；阴影带为 95% 置信区间。

表 3. 四循环模型（PLT4）中每循环行为特征，基于 500 个留出样本平均：每令牌步长 $\delta^{(r)}$、输出分布偏移 $\Delta p^{(r)}$、有效秩以及每个精化循环处的更新对齐余弦 $\theta^{(r)}$。下划线标记每列最大值。

Loop	$\delta^{(r)}$	$\Delta p^{(r)}$	**Eff. rank**	$\cos\theta^{(r)}$
$r = 2$	<u>846</u>	<u>1.75</u>	<u>174.6</u>	<u>−0.72</u>
$r = 3$	464	1.32	172.5	−0.46
$r = 4$	1014	1.58	158.2	0.04

中间令牌 [8, 11]。相比之下，推理（“思考”）模型将显式思维链外化为输出令牌。由于单个额外循环（$R = 2$）是最优工作点（第 4.1 节），我们探究这两种推理通道——显式令牌级思维链与隐式循环级精化——是冗余还是互补。在同一 $R = 2$ 配置下，我们在推理密集型基准上比较指令微调模型（仅隐式循环）与其思考对应模型（在隐式循环之上微调为发出显式推理轨迹的变体）。

表 4. 指令微调与思考模型在最优单个额外循环（$R = 2$).

Model ($R = 2$)	LCB	CRUX	MultiPL-E	FullStackBench	BCB-Hard
Instruct (latent loop only)	35.4	86.9	73.9	47.2	23.7
Thinking (explicit CoT + loop)	**62.3**	**93.5**	**77.8**	**49.9**	**26.4**
Δ	+26.9	+6.6	+3.9	+2.7	+2.7

表 4 显示，在推理繁重任务上，思考变体大幅超越指令微调变体，最显著的是在 LiveCodeBench 上（+26.9 分），远超任一成分单独带来的增益：仅显式思维链不会改善

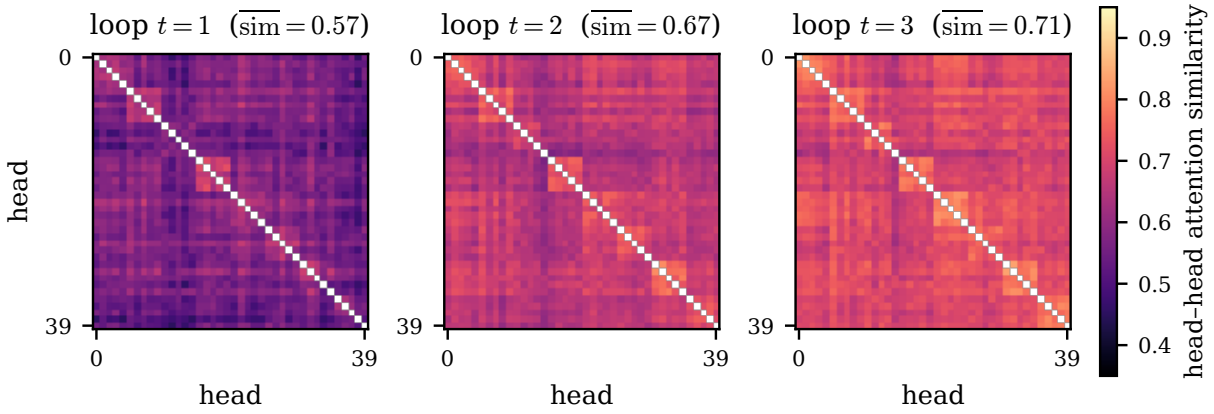


图 4. 循环 × 头间余弦相似度，展示各头注意力分布在循环 $r = 1, 2, 3$ 处的相似性（PLT3，500 个留出样本；对角线上的自相似性已掩蔽）。更亮的单元格表示头冗余度更高；sim 为平均非对角相似度。随着循环进行，头逐渐变得更加冗余。

在这些任务上，非循环模型的表现，而仅靠循环对指令微调模型仅带来个位数的提升。因此，二者的结合是超加性的：联合增益超过了显式推理和潜在循环各自贡献之和。我们将此归因于两种机制在不同粒度上运作：显式思维链将问题分解为中间文本步骤，而潜在循环则细化每个步骤背后的表示，因此通过显式推理轨迹串联循环细化，会复合它们各自的贡献。潜在深度循环与显式推理因此似乎是测试时计算的互补轴，而非替代品，且它们的交互恰好在我们的逐循环分析所确定的单次额外循环操作点上最强。

5. 讨论

第 4 节的逐循环分析揭示了一个一致的模式：循环贡献并非均匀，表示增益与 CLP 偏移所施加成本之间的交互随循环索引变化，从而解释了非单调的性能曲线。

第 2 循环是产生有效精化的主要场所。第二个循环是有效细化的主要场所：在细化循环中，它引入了最连贯的隐藏状态更新、最高的循环间注意力发散以及最大的输出分布偏移，并且有效秩在此达到峰值。第一个循环建立了所有后续循环使用的全局 KV 缓存，因此循环 1 表示的质量为每个后续循环通过冻结的全局注意力分支可用的信息设定了上限。第二个循环随后利用新的局部上下文和全局缓存来细化这些表示，产生最大的净改进。

循环 2 之后：在固定成本下收益递减。超过第 2 轮循环后，模型面临复合约束。有效秩从第 2 轮循环的峰值下降（表示多样性降低），因此共享模块在逐渐低维的输入上运行，削弱了新计算的能力。同时，隐藏状态更新变得振荡而非收敛。此外，固有偏移代价 $\Omega^{(r)}$ 在各轮循环间大致保持不变：CLP 偏移在每一轮都注入相当的位置失配。

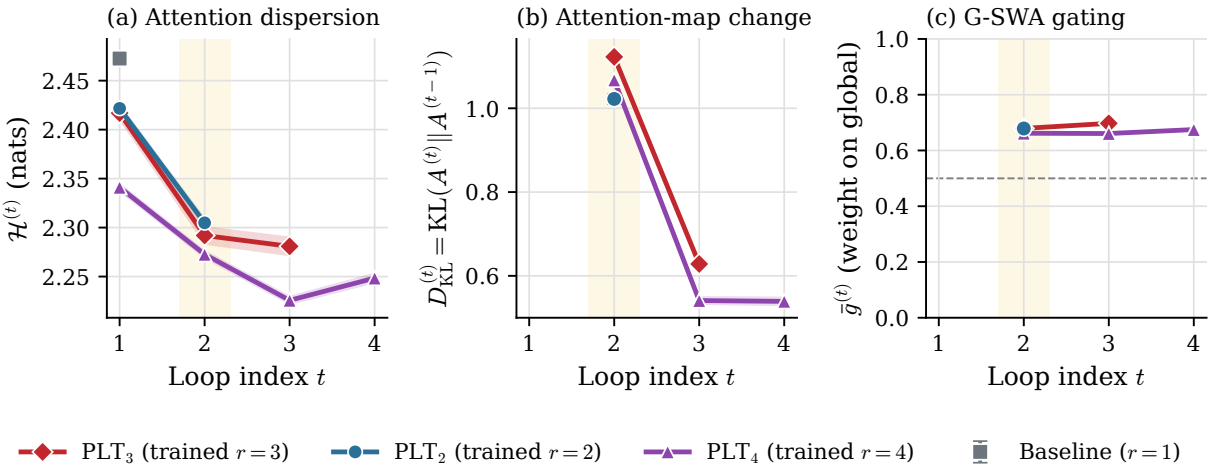


图 5. 平均注意力熵 $\mathcal{H}^{(r)}$（左）、循环间 KL 散度 $D_{\mathrm{KL}}^{(r)}$（中）以及平均 G-SWA 门控 $\bar{g}^{(r)}$（右；全局第 1 轮循环分支的权重）随循环索引 r 的变化（PLT$_2$/PLT$_3$/PLT$_4$）。循环间 KL 散度在第 2 轮循环后急剧下降并保持低位，表明一旦第 2 轮循环完成，注意力路由基本冻结；门控在每一轮循环中始终远高于 0.5。

边界。由于每轮循环的增益迅速缩小，而该偏移代价保持固定，失配在每轮循环净效应中所占份额越来越大，因此超过第 2 轮循环后，代价逐渐超过收益。

循环次数选择的实用指南。这些发现提出了若干实用指南。分析中确定的性能饱和点对应于 PLT 部署的自然运行阈值：$R = 2$ 捕获了主要的细化步骤，同时仅增加一次额外的前向传播。对于更精细的循环次数选择，有效秩轨迹是一种轻量级的逐模型诊断方法，无需穷举搜索：如果有效秩在候选循环处仍在上升（表示多样性尚未饱和），则额外循环可能带来真正的细化；而秩开始下降则表明进入收窄阶段，此后更多循环主要增加固定的 CLP 偏移代价，而没有补偿性收益。

6. 相关工作

6.1. 循环变换器的基础

通用变换器（Universal Transformer，UT）[5] 是循环大语言模型研究的规范起点。UT 在深度方向上共享同一个变换器块，并为其配备逐位置自适应计算时间（Adaptive Computation Time，ACT），这是一种动态停止机制，允许各个词元在不同深度退出循环。在特定条件下，UT 可证明是图灵完备的，这一理论性质通常不适用于固定深度的变换器。循环变换器的可编程计算机解释由 Giannou 等人 [9] 精确化，他们证明循环中恒定数量的编码器层足以模拟通用指令集计算机，包括通过反向传播实现的上下文学习。Yang 等人 [18] 进一步证明，循环变换器在上下文学习基准上与标准变换器表现相当，而使用的参数不到 10%，证实了其实际可行性。

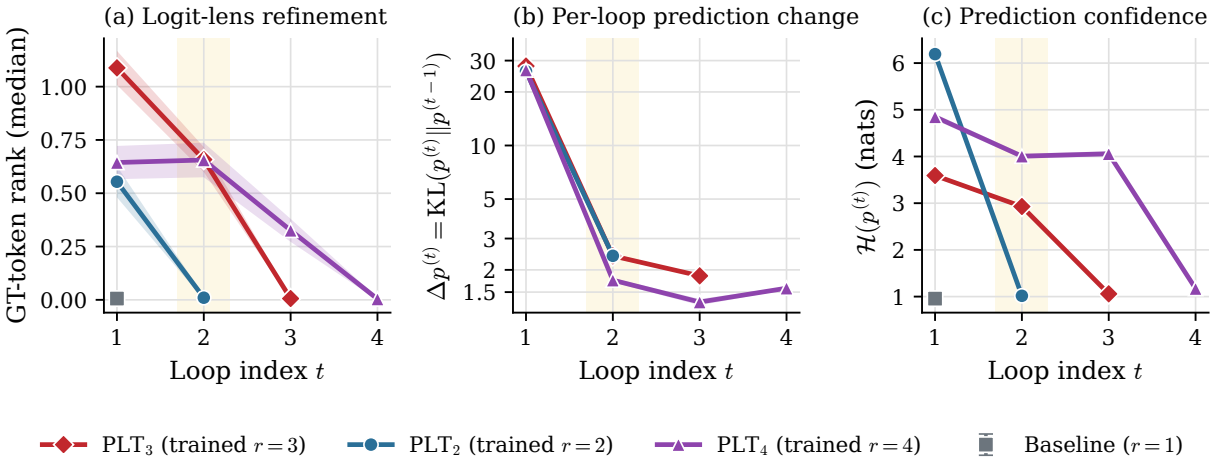


图 6. 逻辑透镜真实词元秩（左）、循环间输出 KL 发散 $\Delta p^{(r)}$（中，对数刻度）和输出熵 $\mathcal{H}(p^{(r)})$（右）随循环索引 r 的变化（PLT $_2$ /PLT $_3$ /PLT $_4$ ）。预测随深度单调锐化，但每循环变化 $\Delta p^{(r)}$ 在循环 2 之后崩溃（最后循环的小幅上升是输出读出而非新的细化），标志着收益递减的开始。

6.2. 通过深度递归实现测试时计算扩展

近期关于循环大语言模型（Looped LLMs）研究的核心动机之一是测试时计算扩展：与其像思维链推理那样生成额外词元，模型可以在更难的输入上运行更多循环，在潜空间中进行隐式多步推理。Geiping 等人 [8] 通过 Huginn-3.5B 大规模展示了这一点，这是一个在 800B 词元上预训练的深度循环变换器，通过在推理时应用最多 50 个循环，在推理基准上取得了改进，实现了等效于 500 亿参数模型的有效计算预算。值得注意的是，这种方法不需要专门的训练数据，并且可以在比思维链方法更小的上下文窗口内运行。然而，这种潜推理范式带来了直接成本：每个额外循环都构成一次通过共享块的完整前向传播，使顺序推理延迟乘以循环次数。更关键的是，标准 KV 缓存实现为每个循环的每一层存储键和值，导致内存增长为 $O(loopcount \cdot L)$，其中块有 L 层，相对于单次前向传播的内存占用，开销是循环次数的倍数。当循环次数过多时，这种开销使得许多架构在设备端或内存受限的部署中不切实际。[15]。

6.3. 内存与延迟降低技术

一些工作旨在保留深度循环的优势，同时降低其推理成本。MELT 通过在各循环间为每一层维护一个共享的 KV 缓存，并通过可学习的门控机制进行更新，从而将循环深度与内存解耦 [15]。PLT 则通过跨循环并行（Cross-Loop Parallelism, CLP）打破循环间的顺序依赖来降低延迟，并结合共享全局 KV 状态和局部当前循环上下文上的门控滑动窗口注意力（Gated Sliding-Window Attention, G-SWA），使内存几乎保持恒定 [16]。LT2 通过用线性或稀疏注意力变体替代二次复杂度的 Softmax 注意力，利用循环进行迭代记忆细化，进一步降低了循环推理的成本 [6]。这些方法提高了循环计算的效率，而我们的工作则关注一个互补的问题：这些效率机制，尤其是 PLT 的 CLP 偏移，如何影响额外循环的有用性。

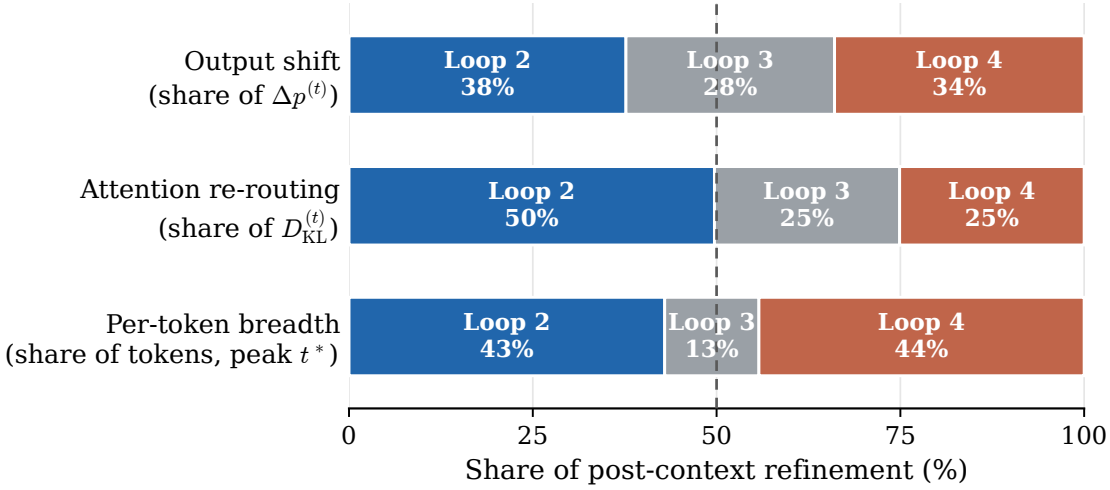


图 7. 后上下文细化如何在 PLT4 的额外循环中分布。每个柱状图将细化循环（$r \geq 2$）所承载的细化拆分为其循环 2、循环 3 和循环 4 的份额，并从三个独立视角进行分析：输出偏移 $\Delta p^{(r)}$、循环间注意力重路由 $D_{\mathrm{KL}}^{(r)}$，以及每个标记的峰值贡献循环 $r^* = \arg\max_r \Delta p^{(r)}$。中间的额外循环（循环 3，灰色）在每个视角上都承载最小的份额：真正的细化集中在循环 2，此时注意力重路由和有效秩也达到峰值，而循环 4 在输出侧的大份额是最终读出而非表征增强（其有效秩最低；图 2c）。（循环 1 构建上下文，被排除以隔离细化。）

6.4. 架构变体

除了以效率为导向的设计，另一条工作线探索更丰富的循环架构。一些方法通过添加轻量级的循环特定适配来放宽严格的权重共享，例如用于将预训练大语言模型转换为递归变换器的深度方向低秩适配器 [1]。其他方法重构循环计算本身，包括具有开始-中间-结束划分和用于循环间混合的超连接的超环变换器 [20]，具有上下文锚定循环核心的 CART，该核心交叉关注冻结的预计算上下文张量 [2]，以及具有在不同循环时间尺度上运行的快慢模块的 HRM-LM[10]。额外的变体通过专家混合前馈层 [4]、具有吸引子模块的不动点细化 [7]，或将标准大语言模型训练后转换为循环编码器-推理器-解码器架构 [13] 来增加容量或稳定性。这些工作拓宽了循环变换器的设计空间，而我们的关注点是互补的：我们研究循环计数如何影响高效 PLT 架构的行为以及其性能为何饱和。

6.5. 缩放定律与表征动态

近期研究已开始探讨循环模型如何随规模扩展，以及其内部表征如何随递归深度演化。缩放律分析表明，增加循环次数带来的收益递减：Schwethelm 等人 [14] 估计，将一个模块循环 r 次在验证损失上仅相当于 $r^{0.46}$ 个独立参数，远低于完全等同于添加新层。互补研究考察了递归表征动态，发现循环更新可能变得更小、更正交，或在多个时间尺度上呈现结构化 [12]。其他工作强调稳定性是核心局限：Yang 等人 [19] 表明性能可能在中间循环深度达到峰值后崩溃，并提出不动点正则化以稳定递归计算。在可解释性方面，先前分析探究了更深递归是否对应于有意义的潜在推理或类似自然语言的中间

计算，通常发现混合证据和表征退化的迹象 [3, 11]。我们的工作在精神上与这些表征动态研究最为接近，但特别聚焦于 PLT：我们分析其效率机制如何改变跨循环的增益-成本曲线，以及为何在低循环次数时出现饱和。

7. 结论

循环变换器（Looped Transformers）提供了一种在不增加参数数量的情况下扩展潜在计算的吸引人机制，但其在循环次数增加时的行为仍未被充分理解。本文从增益-成本视角研究 PLT 中的这一问题：每次额外循环可能提供有用的细化，但 CLP 偏移也在每个循环边界引入结构性位置不匹配。我们的受控逐循环分析表明，第二次循环是产生有效细化的主要来源，在隐藏状态、注意力路由和输出分布上产生有意义的变化，而后续循环带来的更新递减且越来越振荡。由于 CLP 引起的不匹配在边际循环增益缩小时大致保持不变，额外循环最终变得无效，这解释了 PLT 为何在低循环次数时饱和。这些发现为循环次数选择提供了基于可解释性的诊断方法，无需穷举基准扫描，并指出了未来方向，如自适应偏移机制、动态循环分配，以及更深入理解潜在递归与显式思维链推理之间的相互作用。

参考文献

1 Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. *arXiv preprint*, 2024. doi: 10.48550/arxiv.2410.20672.

2 Chad A. Capps. CART: Context-anchored recurrent transformer – a parameter-efficient architecture with learned stability. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2606.01495.

3 Guanxu Chen, Dongrui Liu, and Jing Shao. Loop as a bridge: Can looped transformers truly link representation space and natural language outputs? *arXiv preprint*, 2026. doi: 10.48550/arXiv.2601.10242.

4 Robert Csordas, Kazuki Irie, Jurgen Schmidhuber, Christopher Potts, and Christopher D. Manning. MoEUT: Mixture-of-experts universal transformers. *arXiv preprint*, 2024.

5 Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. *arXiv preprint*, 2018. doi: 10.48550/arXiv.1807.03819.

6 Chunyuan Deng, Yizhe Zhang, Rui-Jie Zhu, Yuanyuan Xu, Jiarui Liu, T. S. Eugene Ng, and Hanjie Chen. LT2: Linear-time looped transformers. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2605.20670.

7 Jacob Fein-Ashley and Paria Rashidinejad. Solve the loop: Attractor models for language and reasoning. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2605.12466.

8 Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint*, 2025.

9 Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. *arXiv preprint*, 2023. doi: 10.48550/arXiv.2301.13196.

10 Sang-Il Han. Hierarchical vs. flat iteration in shared-weight transformers. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2604.14442.

11 Wenquan Lu, Yuechuan Yang, Kyle Lee, Yanshu Li, and Enqi Liu. Latent Chain-of-Thought? decoding the depth-recurrent transformer. *arXiv preprint*, 2025. doi: 10.48550/arxiv.2507.02199.

12 Francesco Pappone, Donato Crisostomi, and Emanuele Rodola. Two-scale latent dynamics for recurrent-depth transformers. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2509.23314.

13 Taekhyun Park, Yongjae Lee, Dohee Kim, and Hyerim Bae. LoopUS: Recasting pretrained LLMs into looped latent refinement models. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2605.11011.

14 Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How much is one recurrence worth? Iso-Depth scaling laws for looped language models. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2604.21106.

15 Victor Conchello Vendrell, Arnau Padres Masdemont, Niccolo Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. Memory-efficient looped transformer: Decoupling compute from memory in looped language models. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2605.07721.

16 Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel loop transformer for efficient test-time computation scaling. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2510.24824.

17 Jian Yang, Xianglong Liu, Weifeng Lv, Ken Deng, Shawn Guo, Lin Jing, Yizhi Li, Shark Liu, Xianzhen Luo, Yuyu Luo, et al. From code foundation models to agents and applications: A comprehensive survey and practical guide to code intelligence. *arXiv preprint arXiv:2511.18538*, 2025.

18 Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. *arXiv preprint*, 2023. doi: 10.48550/arXiv.2311.12424.

19 Xiao-Wen Yang, Ziyu Han, Xi-Hua Zhang, Wen-Da Wei, Jie-Jing Shao, Lan-Zhe Guo, and Yu-Feng Li. Stabilizing recurrent dynamics for test-time scalable latent reasoning in looped language models. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2605.26733.

20 Abbas Zeitoun, Lucas Torroba-Hennigen, and Yoon Kim. Hyperloop transformers. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2604.21254.

A. Forward-Pass Pseudocode

Algorithm 1 gives the PLT forward pass ($w = 64$ throughout).

Algorithm 1 PLT Forward Pass ($w = 64$)

Input: tokens x, loop count R
Output: logits y

$\mathbf{h}^{(0)} \leftarrow \mathrm{Embed}(x)$
for $r = 1$ **to** R **do**
  if $r = 1$ **then**
    $\mathbf{h}^{(1)} \leftarrow f_\theta(\mathbf{h}^{(0)})$ ▷ standard full-context attention
    $K_{\text{share}}, V_{\text{share}} \leftarrow \mathrm{KV}(\mathbf{h}^{(1)})$ ▷ loop-1 KV cache, shared with all subsequent loops
  else
    $B^{(r)} \leftarrow \mathrm{Embed}(x) + \mathrm{shift}(\mathbf{h}^{(r-1)})$ ▷ CLP: right-shift by 1 position
    $y_{\text{global}} \leftarrow \mathrm{Attn}(Q(B^{(r)}), K_{\text{share}}, V_{\text{share}})$
    $y_{\text{local}} \leftarrow \mathrm{SWAttn}(Q(B^{(r)}), K(B^{(r)}), V(B^{(r)}), 64)$
    $g^{(r)} = \sigma(f_{\text{gate}}(\mathrm{RMSNorm}(B^{(r)})))$. ▷ G-SWA fusion (Equation 1)
    $\mathbf{h}^{(r)} \leftarrow \mathrm{FFN}(\tilde{y})$
  end if
end for
$y \leftarrow \mathrm{Head}(\mathbf{h}^{(R)})$

B. 模型架构配置

表 5. 所有实验中使用的基座模型配置。

Hyperparameter	**Value**
Layers L	14
Hidden size d	5120
Attention heads H	40
KV groups (GQA)	8
Head dimension	128
FFN intermediate size	27,648
Activation	SwiGLU
Normalization	RMSNorm ($\epsilon = 10^{-5}$)
Position embedding	RoPE (base 5×10^5)
Attention	Flash Attention, no dropout
QK LayerNorm	no
Bias in linear layers	no
Precision	bf16
Vocabulary size	76,800
Total parameters	≈7B
Training tokens	18T
Window size w	64 (fixed)
CLP offset	enabled
Loop counts R analyzed	1, 2, 3, 4

逐循环可解释性分析（第 4 节）在所有循环次数上进行，$R \in \{1, 2, 3, 4\}$。

C. 预训练代码-数据构成

预训练语料库按 1:1 的文本与代码标记比例进行平衡（第 3.1 小节）。表 6 按编程语言细分了代码部分：按标记份额列出最大的十种语言，其余 93 种语言归入“其他”。份额仅基于代码标记计算。

表 6. 18T 标记预训练混合中代码部分的构成，以所有代码标记的标记份额表示。前 10 种语言单独列出；其余 93 种语言归为一组。

Language	**Token share (%)**
Java	10.3
Python	10.1
JavaScript	9.4
Markdown	8.7
TypeScript	8.3
C	5.2
C++	5.0
PHP	4.7
C#	4.0
HTML	3.7
Others (93 languages)	30.5